2026

sellsell marketplace updates: Audit

SHAZ EJ

Here is the multi-agent plan for SellSell / PickPick, based on your audit. The key point: **do not build agents first. Build the agent infrastructure first.** Your audit says the foundation exists — Ollama, Qdrant, Redis, BullMQ, Prisma — but there is no orchestrator, no per-agent jobs, no AgentLog table, and no product-level status tracking yet.

Multi-Agent Plan

Phase 0: Fix the dangerous parts first

Before multi-agent, fix production safety.

1. New listings must be created as:

```
status: "pending"
moderationStatus: "pending"
embeddingStatus: "pending"
```

Not `active`.

2. Listing becomes `active` only after required agents pass.
3. Fix SSRF image URL validation.
4. Remove production `DEV_AUTH_BYPASS`.
5. Reduce Ollama search timeout from 150 seconds to around 30 seconds.
6. Stop fire-and-forget moderation. Put it inside BullMQ.

This is the safety foundation.

Phase 1: Add database fields for agents

Add these fields to `Product`:

```
moderationStatus
moderationScore
moderationReason

embeddingStatus
embeddingError
lastIndexedAt

imageValidationStatus
imageValidationReason

categoryAgentConfidence

spamScore
spamReason
```

```
duplicateScore
duplicateOfId

aiDescriptionStatus

agentPipelineStatus
lastAgentRunAt
```

Add new table:

```
AgentLog
```

Fields:

```
id
productId
agentType
status
inputJson
outputJson
error
latencyMs
modelUsed
startedAt
finishedAt
createdAt
```

This table is very important. Without it, you cannot debug agents.

Phase 2: Create the Agent Orchestrator

Create:

```
src/lib/agents/orchestrator.ts
```

Its job:

```
When product is created:
1. Create product as pending
2. Start agent pipeline
3. Run required agents
4. Store every result
5. Decide final status
```

Example:

```
Product Created
   ↓
Publishing Agent
   ↓
Image Validation Agent
   ↓
Category Agent
```

```
   ↓
Spam Detection Agent
   ↓
Duplicate Detection Agent
   ↓
Data Structuring Agent
   ↓
AI Description Agent
   ↓
Embedding Agent
   ↓
Final Approval Agent
```

Phase 3: Create separate BullMQ jobs

Do not keep everything inside `/api/products`.

Create agent jobs:

```
listing-agent-pipeline
image-validation-agent
category-agent
spam-agent
duplicate-agent
data-structuring-agent
ai-description-agent
embedding-agent
final-approval-agent
```

Files:

```
src/jobs/agents/listing-pipeline.job.ts
src/jobs/agents/image-validation.job.ts
src/jobs/agents/category.job.ts
src/jobs/agents/spam.job.ts
src/jobs/agents/duplicate.job.ts
src/jobs/agents/data-structuring.job.ts
src/jobs/agents/description.job.ts
src/jobs/agents/embedding.job.ts
src/jobs/agents/final-approval.job.ts
```

Phase 4: Seller-side agents

1. Publishing Agent

Purpose:
Validate seller input and create listing as pending.

Rules:

- price must come from user

- seller title/description has priority
- AI cannot override confirmed user data
- product is not active yet

---

2. Image Validation Agent

Purpose:
Check uploaded image.

It should detect:

- is it a real product image?
- does image match category?
- is it spam/logo/screenshot?
- is it unsafe?
- is image too blurry?

Output:

```
{
  "approved": true,
  "confidence": 0.91,
  "reason": "Image appears to match a black Mercedes vehicle"
}
```

---

3. Category Agent

Purpose:
Confirm or correct category.

Input:

- title
- description
- image analysis
- seller category if available

Output:

```
{
  "categorySlug": "vehicles",
  "confidence": 0.96,
  "reason": "Title and image indicate a car"
}
```

If confidence is low, send listing to admin review.

---

4. Spam Detection Agent

Purpose:
Detect scam/spam listings.

Checks:

- repeated phone numbers
- suspicious keywords
- unrealistic price
- duplicate text
- banned terms
- Gulf Arabic scam patterns

Output:

```
{
  "spamScore": 0.12,
  "status": "passed",
  "reason": "No suspicious pattern found"
}
```

---

5. Duplicate Detection Agent

Purpose:
Stop repeated listings.

Checks:

- same seller posted same title
- same phone number
- similar image
- similar embedding in Qdrant
- same price/category/image pattern

Output:

```
{
  "duplicateScore": 0.82,
  "possibleDuplicateProductId": "...",
  "status": "needs_review"
}
```

---

6. Data Structuring Agent

Purpose:
Convert listing into structured marketplace data.

Example seller writes:

```
Mercedes S400 black 2020 good condition
```

Agent extracts:

```
{
  "brand": "Mercedes",
  "model": "S400",
  "year": 2020,
  "color": "black",
  "condition": "good",
  "category": "vehicles"
}
```

Store in:

- `ProductAttributeValue`
- `VehicleDetails`
- product smart fields
- Qdrant payload

---

7. AI Description Agent

Purpose:
Improve poor descriptions.

Only run if description is too short.

It should create:

- English description
- Arabic description
- search-friendly keywords

But it must not invent facts.

---

8. Embedding / Qdrant Agent

Purpose:
Index product in Qdrant.

It should embed:

- seller title
- seller description
- Arabic title/description

- AI structured attributes
- keywords
- category
- brand/model/color/year

Then set:

```
embeddingStatus = "indexed"
lastIndexedAt = now()
```

If it fails:

```
embeddingStatus = "failed"
embeddingError = error.message
```

---

9. Final Approval Agent

Purpose:
Decide if product becomes active.

Rules:

```
If moderation passed
AND image validation passed
AND spamScore is safe
AND duplicateScore is safe
AND embedding indexed
→ status = active

If small issue
→ status = pending_review

If dangerous issue
→ status = rejected
```

Phase 5: Buyer-side agents

Your buyer search already has many pieces. Now split them cleanly.

1. Buyer Intent Agent

Understands the buyer request.

Example:

```
I want black Mercedes 2020 under 5000 KWD
```

Output:

```
{
  "intent": "buy_vehicle",
  "brand": "Mercedes",
  "color": "black",
  "year": 2020,
  "maxPrice": 5000,
  "currency": "KWD"
}
```

---

2. Keyword Extraction Agent

Creates search keywords:

```
{
  "keywords": ["Mercedes", "S400", "black", "2020", "sedan"],
  "arabicKeywords": ["مرسيدس", "أسود", "٢٠٢٠"]
}
```

---

3. Search Planning Agent

Decides how to search:

```
Use Qdrant semantic search
Apply PostgreSQL filters for price/category/status
Fallback to SQL if Qdrant fails
```

---

4. Matching Agent

Runs Qdrant hybrid search.

Must use:

- dense vector
- BM25 sparse
- payload filters
- active listings only

---

5. Ranking Agent

Ranks results by:

- semantic score
- price match
- location
- freshness
- seller quality

- exact attribute match
- click/contact behavior

---

6. Result Explanation Agent

Explains why the result matched.

Example:

```
I found this because it matches Mercedes, black color, 2020 year, and your price range.
```

Arabic first.

---

7. Learning Agent

Tracks:

- search query
- shown listings
- clicks
- WhatsApp/call actions
- ignored listings

Later this improves ranking.

Recommended implementation order

Week 1

Fix product publishing flow:

```
active → pending first
moderation moved to queue
embeddingStatus added
moderationStatus added
AgentLog added
```

Week 2

Build orchestrator:

```
src/lib/agents/orchestrator.ts
src/jobs/agents/listing-pipeline.job.ts
```

Run only 3 agents first:

- moderation
- embedding
- final approval

Week 3

Add:

- image validation
- category confidence
- spam score

Week 4

Add:

- duplicate detection
- AI description
- data structuring

Week 5

Clean buyer search:

- Buyer Intent Agent
- Search Planning Agent
- Ranking Agent split into separate services

Final architecture

```
Seller uploads listing
        ↓
Product saved as pending
        ↓
Agent Orchestrator starts
        ↓
Agents analyze listing in background
        ↓
Results stored in AgentLog
        ↓
Product fields updated
        ↓
Qdrant indexed
```

```
        ↓
Final Approval Agent decides:
        active / pending_review / rejected
```

This is the correct path. Not “many AI calls everywhere” — controlled agents, tracked results, background jobs, and admin visibility.